The 96/4 split: answers to your questions, and a revised fourth change

Andreas Kohl · Sequentia testnet · 2026-07-01 · follows the 2026-06-28 memo (commit `23b1be0b1`; file and line references are to that commit, re-verified against the code)

1. Your reconstruction is correct

Bitcoin went A, then B, then back to A. Sequentia produced chain A1 anchored to A; when Bitcoin moved to B the anchor watcher invalidated A1 and the committee produced B-anchored blocks; when Bitcoin returned to A those were invalidated in turn, and the network raced between restoring A1 and minting a fresh A2. Both outcomes happened at once on different nodes: that is the 96/4. The Sequentia blocks anchored to B are discarded outright (their anchor left Bitcoin's best chain), and A1, not A2, must survive, for exactly the reason you gave: A1 is the certified block whose transactions (for example the Sequentia leg of an atomic swap whose Bitcoin leg sits in A) were already exposed as final; A2 was built later and may omit them. The three shipped changes make every node that holds A1 restore it verbatim. What they do not do is stop other nodes from minting and certifying a fresh A2 in the window before restoration completes; that is the remaining gap, addressed in section 6.

"Comes later in human time" is the right intuition, but no node can verify wall-clock order globally; the enforceable form of your rule is the prevention guard in section 6.

2. The committee of A1 and A2 is the same

Confirmed, with one delta from the formula you wrote, and the delta is in the safe direction. Committee and leader selection are seeded by `SHA256(parent block's Bitcoin anchor hash || child height)` (`pos.cpp:67-80, 292-305`). The parent Sequentia block hash is deliberately NOT an input: the seed must be fixed before the round starts, and using the parent's anchor instead of the parent's own hash removes the last-revealer grinding channel (the producer of the parent cannot re-roll the next committee by tweaking its own block). Since A1 and A2 extend the same surviving parent, their seed is identical: same eligible committee, and the stake weights are evaluated against the same parent state (`validation.cpp:2138-2146`).

Two qualifications. The leader need not be the same node: under VRF sortition there is no single leader per height; any eligible staker whose time slot has opened may propose (`validation.cpp:2202-2209`), so a later A2 can validly carry a different leader. And a certificate may name ANY quorum subset (51 of the roughly 100 eligible, `validation.cpp:2293-2294`), so A1's and A2's signer sets can differ. A2 referencing a later Bitcoin block on the A chain affects only the seed of the NEXT height.

3. Honest nodes signed both blocks, and that is unavoidable under anchoring

Your first question: this was not an adversarial test, and no attacker was involved. Two quorum certificates from the same committee share signers in practice (two majorities drawn from the same roughly hundred-member committee), and the shared signers acted honestly. The sequence for such a node: it countersigned A1; Bitcoin left A, so it invalidated A1 (anchoring is supreme; this is correct); the height became vacant in its view and its round state reset (`pos_producer.cpp:536-555`). Signing a different block at the same height after that is by design; the same re-sign path is what lets a round move to a new leader, so it cannot simply be forbidden. When Bitcoin returned to A, restoring A1 takes one anchor-watcher cycle (a poll tick plus a live bitcoind check), while proposing fresh is immediate, because the slot deadlines at the rolled-back height had long passed. That night's CPU overload stretched the watcher cycle, so part of the committee certified a fresh A2 before the rest finished restoring A1. Both blocks anchor to the A chain, so anchoring cannot arbitrate, and each side finalized whichever it completed first.

> The design consequence: committee equivocation here is a byproduct of correctly following a Bitcoin reorg of a reorg. Punishing double-signing would punish honest behavior; the fix belongs in production policy (section 6), not in slashing.

4. The convergence rule is implemented, and it already refuses posterior reorgs

Both halves of the theoretical paper's rule are in fork choice (`validation.cpp:135-150`): among competing blocks, more countersignatures wins; at equal count, lower VRF wins; then first seen. So the lower-VRF half you wanted checked is present.

It is deliberately subordinate to immediate finality. The acceptance gate rejects any fork at or below the finalized block, explicitly including a same-height competitor carrying MORE signatures (`validation.cpp:4314-4318`), and the fork-choice comment states that a finalized block must never be reorged by a Sequentia-internal competitor. That is your posterior-corruption concern, already enforced: signatures accumulated after the fact cannot move a finalized block, so buying keys from past stakers buys nothing at certified heights. The comparator therefore arbitrates only among blocks that are not yet immediately final, which is exactly the escaping-stall regime we agreed on.

> **Withdrawal.** I withdraw the fourth-change shape from my previous memo (yield to a strictly-better-certified same-height sibling). You are right that it reopens the posterior channel; the code's current refusal is the correct baseline, and the revised proposal below never compares certificates at the same height.

5. Escaping stall: semantics confirmed

As you described: a sub-quorum block is acceptable only when its Bitcoin anchor is at least 3 blocks past its parent's (`pos.h:338-352`); it never advances the immediate-final point (`validation.cpp:3141-3153`); it becomes final only when a quorum-certified block lands on top; a full-quorum rival beats it in fork

choice; each further sub-quorum block needs another 3 Bitcoin blocks of progress; and the committee is re-drawn every height, since the height is a seed input. Your operational note is right as well: a DEX should treat a payment in a sub-quorum block as unconfirmed until a quorum block covers it. We will surface "in an immediately final block" through the node RPC so the trading stack can do exactly that.

6. The revised fourth change

Change 4a. Prevention (node-local, no consensus rule change) IMPLEMENTING NOW

Your sentence "nodes shouldn't create, broadcast, vote and sign another block at the same height" becomes a production rule: while a node holds a quorum-certified block at height h that is awaiting, or has received, a favorable anchor verdict (its anchor is back on Bitcoin's best chain, or the watcher has not yet re-checked it since the parent chain moved), it neither proposes nor backs any other block at h; it waits for the restoration, with a short bounded patience (about one block interval) so production can never deadlock. If the verdict is negative (Bitcoin genuinely stays away), production proceeds exactly as today. This closes the race from section 3 at its source, changes no consensus rule (old and new nodes interoperate), and never resists an anchor invalidation. It will not be deployed until you ack this memo.

Change 4b. Reconciliation (consensus rule) YOUR CALL

Prevention shrinks the window but cannot make splits impossible (a partition, or two certificates completing near-simultaneously). Once a split exists, first-seen alone never converges: each side is finalized and stuck. Proposed rule: a node abandons its finalized block ONLY for a rival branch that

1. forks no lower than the latest Bitcoin checkpoint,
2. carries anchors that are all on Bitcoin's current best chain, and
3. contains a quorum-certified block at a height STRICTLY ABOVE the node's finalized height.

Certificates at the same height are never compared, so the posterior channel stays closed: to move anyone, an attacker must win fresh quorums at new heights inside the checkpoint band, which requires controlling current committees, not old keys. In the live 96/4 this converges the minority at once (the majority branch had quorum blocks above the split immediately) and the majority never yields (a branch that only 4 of 100 nodes consider valid can never produce another quorum certificate).

Residual case for you: a near-perfect half-and-half split, where neither side can reach quorum again. Change 4b never fires there. Either we leave it to operators (it needs an almost exactly simultaneous certification race, which we have never observed), or we allow your same-height rule (more signatures, then lower VRF) as a last resort, gated on N Bitcoin blocks passing with no quorum progress on either side.

Footnote: today the finality gate acts only at header acceptance; a rival that entered a node's index before that node finalized can still win through the comparator. The 4b predicate can also be enforced at chain-activation time to close that residual window; cheap to include if you want it.

Questions:

1. Do you approve Change 4a (prevention)? It deploys to the testnet on your ack.

2. Do you approve the Change 4b predicate: yield only to checkpoint-floored, fully anchor-valid quorum progress strictly above the finalized height, never a same-height comparison?

3. The symmetric residual: operator-only, or a last-resort same-height tie-break (more signatures, then lower VRF) after N Bitcoin blocks without quorum progress? If the latter, what N?

Full code citations for every claim are in `doc/sequentia/anchor-reorg-of-reorg-recovery-design.md` (updated alongside this memo) in the SequentiaByClaude repository.